NET (BMEVITMAC09): three mock midterms with solutions

Mock papers written by us, not official papers. Format (6 open questions, 5 points each, 30 points, pass mark 12/30, on paper) and scope (sessions 1 to 6, slides hta1 to hta6p) as announced by the lecturer and reported by Elisha. Real midterm: Thursday 2026-10-29, 8:15, IE218. Write your answers in the boxes; the solutions part repeats each question with the box filled in, followed by the marking scheme and the usual pitfalls.

Paper A

6 questions, 5 points each, 30 points, pass mark 12/30. Justify every answer.

1. IPv4 addressing (5 points)

inspired by: 2018 midterm, "same subnet"; hta2p p. 12

A host is configured as 172.20.83.140/21.

(a) Give the netmask in dotted decimal notation. (1 pt)

(b) Compute the network address, the broadcast address and the number of assignable host addresses. (2 pts)

(c) For each address below, say whether it is valid, whether it is on the same subnet as the host, and whether it can be assigned to another host of that subnet: 172.20.86.17; 172.20.79.254; 172.20.88.1; 172.20.87.255; 172.20.80.0; 172.20.84.300. (2 pts)

2. Longest prefix match (5 points)

inspired by: 2018 midterm, routing table; hta2p p. 13 and 14

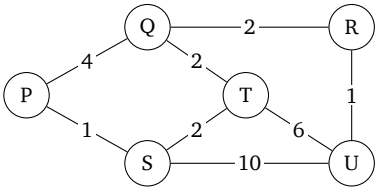
Router R uses the routing table on the left.

Prefix	Interface
84.112.0.0/12	eth0
84.120.0.0/13	eth1
84.124.0.0/14	eth2
84.126.64.0/18	eth3
0.0.0.0/0	eth4

(a) Give the outgoing interface for packets destined to 84.127.70.5, 84.126.100.1, 84.119.3.3 and 84.128.0.1. For each one, list the matching lines. (4 pts)

(b) An administrator wants to add the line 84.126.100.0/18 towards eth5. Why is it written incorrectly, and which network does it actually denote? (1 pt)

3. Link state: Dijkstra's algorithm (5 points) inspired by: 2017 topic list, no. 16; hta4p p. 8 to 14



Link costs are shown on the graph.

(a) Run Dijkstra's algorithm from P. Give a table: at each step, the node taken from the priority queue and, for every other node, the tentative distance and the previous node. On a tie, take the smallest letter. (3 pts)

(b) Derive the FIB of P: destination, cost, next hop. (1 pt)

(c) The Q-R link fails. Give the new costs from P to R and to U. According to the course, which family reconfigures faster after a link failure: link state or distance vector? (1 pt)

4. TCP Reno congestion control (5 points) inspired by: 2017 topic list, no. 33; hta5p p. 8 to 15

Conventions: cwnd and ssthresh in MSS, one round per round trip, cwnd segments sent per round. Slow Start (SS): cwnd doubles every round, **capped at ssthresh** (if doubling would exceed ssthresh, cwnd takes the value ssthresh), then Congestion Avoidance (CA). CA: cwnd +1 per round and ssthresh = cwnd. Three duplicate ACKs: cwnd and ssthresh both become half of cwnd, stay in CA. Timeout: ssthresh = cwnd/2, cwnd = 1, back to SS. A loss detected at the end of a round changes the next round; adv_wnd never limits the sender. In round 1, cwnd = 1 and ssthresh = 12. The sender receives three duplicate ACKs at the end of round 9 and has a timeout at the end of round 14.

(a) Give cwnd, ssthresh and the phase (SS or CA) for rounds 1 to 19. (3 pts)

(b) How many segments are sent in total during rounds 1 to 19? (1 pt)

(c) Why does Reno wait for three duplicate ACKs rather than one, and why does it treat a timeout more severely? (1 pt)

5. Serialization: XML to JSON and YAML (5 points) inspired by: hta1p p. 26, "Possible exam question"

A DHCP server exports a lease in the XML format shown on the left.

```
<lease>
  <ip>192.168.10.57</ip>
  <hostname>labo-pc7</hostname>
  <duration>86400</duration>
  <options>
    <router>192.168.10.1</router>
    <dns>192.168.10.2</dns>
    <dns>192.168.10.3</dns>
  </options>
</lease>
```

(a) Write the same content in JSON. (2 pts)

(b) Write it in YAML. (2 pts)

(c) Why does the course recommend `JSON.parse()` rather than `eval()` to read JSON in JavaScript? (1 pt)

6. Multiple access, duplexing and token bucket (5 points) inspired by: 2018 midterm, Q1; hta1 p. 19 to 21; hta1p p. 18 and 19

A radio operator has two 1.6 MHz bands, one for the uplink and one for the downlink. Each band is split into 200 kHz carriers, and each carrier into frames of 8 time slots. A call uses one slot on an uplink carrier and one slot on a downlink carrier.

(a) Define FDMA, TDMA and CDMA in one sentence each. Which multiple access and duplexing techniques does this operator combine? What is the maximum number of simultaneous calls? (2 pts)

(b) A second operator has a single 3.2 MHz band, with the same carriers and frames, and uses Time Division Duplex (TDD). What is the principle of TDD? What is the maximum number of symmetric calls? Which constraint does time sharing impose on the equipment? (1 pt)

(c) At the network entrance, a client's flow is metered by a token bucket (one token = one bit). Contract: average bitrate 4 Mbit/s, maximum bitrate 10 Mbit/s, bursts of 0.3 s. Give K and B with the design rule of the course. The bucket is full and the client sends at 10 Mbit/s: after how long is the bucket empty, and what happens then to the traffic above 4 Mbit/s? What would a leaky bucket do instead? (2 pts)

1. Variable length subnetting (5 points) inspired by: 2019 practice set, ex. 6 (splitting a /20); hta2p p. 7

A company owns the block 10.62.96.0/20. Needs: Production 1000 hosts, Offices 450, Laboratory 120, Guest Wi-Fi 60, and two point-to-point links between routers (two addresses each).

(a) Which prefix length does each need require? (1 pt)

(b) Allocate the subnets from the start of the block, largest first. For each one, give network/prefix and broadcast address. (3 pts)

(c) What is the largest aligned subnet still free in the block? Which single prefix does the company announce to its provider? (1 pt)

2. CIDR aggregation (5 points) inspired by: 2018 midterm, Q10; hta2p p. 14

A router has the table on the left and no default route.

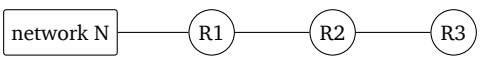
Prefix	Next hop
185.40.16.0/22	A
185.40.20.0/22	A
185.40.24.0/21	A
185.40.32.0/21	A
185.40.40.0/22	B
185.40.44.0/22	A
185.40.48.0/20	A

(a) Give the address range of each line (the 3rd octet is enough). (1 pt)

(b) Reduce the table to the minimum number of lines by exact merging: the new table covers exactly the same addresses, every address keeps its next hop, and no line may overlap another. (2.5 pts)

(c) Now a more specific line may override a wider one (longest prefix match). Reduce the table to three lines without changing the output of any address. Why can't everything be summarized as 185.40.0.0/18 towards A? (1.5 pts)

3. Distance vector: count to infinity (5 points) inspired by: 2018 midterm, Q3; 2020 exams; hta4p p. 4 to 6



The routers run a RIP-like distance vector protocol: hop count metric, R1 is directly connected to N with distance 1, infinity is 16.

(a) After convergence, give the distance and next hop towards N in R1, R2 and R3. What does R2's advertisement contain about N, and what is not in it? (1.5 pts)

(b) The R1-R2 link goes down. R2 marks N unreachable, but receives R3's periodic update before it has advertised the failure; updates then alternate, R3 then R2. Describe the first four updates (router, distance, next hop). After how many updates do both R2 and R3 consider N unreachable? What happens meanwhile to packets towards N? (2 pts)

(c) Explain split horizon and poisoned reverse on this example: what does R3 advertise to R2 about N in each case, and what becomes of scenario (b)? (1.5 pts)

4. NAT (5 points) inspired by: no past paper; hta5 p. 18 and 19

Home network 192.168.0.0/24; NAT gateway with internal address 192.168.0.1 and external address 81.183.20.9. Hosts 192.168.0.23 and 192.168.0.42 each open a TCP connection to server 145.236.10.5, port 443, both with source port 51000. The gateway picks external ports 40100 (for .23) and then 40101 (for .42).

(a) For the connection of host .23, give source and destination (address and port) on the four legs: host to gateway, gateway to server, server to gateway, gateway to host. (2 pts)

(b) Write the gateway's association table. Why can't the gateway keep port 51000 for both connections? Reason with the 5-tuple. (1.5 pts)

(c) An internal web server, 192.168.0.50 port 8080, must be reachable from the Internet. Why doesn't the NAT described here allow it, and which solutions does the course mention? (1.5 pts)

5. Serialization: XML to JSON and YAML (5 points) inspired by: hta1p p. 26, "Possible exam question"

A router configuration is given in XML on the left.

```
<router>
  <name>edge1</name>
  <ospfArea>0.0.0.0</ospfArea>
  <interfaces>
    <interface><name>eth0</name>
      <address>10.0.0.1/30</address>
      <mtu>1500</mtu></interface>
    <interface><name>eth1</name>
      <address>192.168.5.1/24</address>
      <mtu>9000</mtu></interface>
  </interfaces>
  <protocols>
    <protocol>ospf</protocol>
    <protocol>bgp</protocol>
  </protocols>
</router>
```

(a) Write the same content in JSON. (2 pts)

(b) Write it in YAML. (2 pts)

(c) The course also presents TLV (Type, Length, Value) encoding. Why can a reader written for an older version of a TLV protocol still read a message that contains a new type? (1 pt)

6. Web: HTTP, TLS and QUIC; quality of service (5 points) inspired by: no past paper; hta6 p. 8 to 15; hta1p p. 16

A browser opens `https://www.example.org/index.html` for the first time.

(a) List in order the exchanges needed until the page is received with HTTP/1.1: protocol, transport, port, role of each exchange. (2 pts)

(b) The server presents an X.509 certificate. Under which two conditions does the browser accept it? What does a self-signed certificate allow, and what does it not allow? (1 pt)

(c) What changes with HTTP/3 over QUIC: transport, handshakes, encryption, mobility? (1 pt)

(d) Outside the Web, session 1: the course distinguishes IntServ, DiffServ and best effort. How do they differ? Which one describes how the Internet runs, and why? (1 pt)

1. Session 1: Protocol stack and statistical multiplexing (5 points) inspired by: hta1p p. 3 to 6 and 20; hta5 p. 7

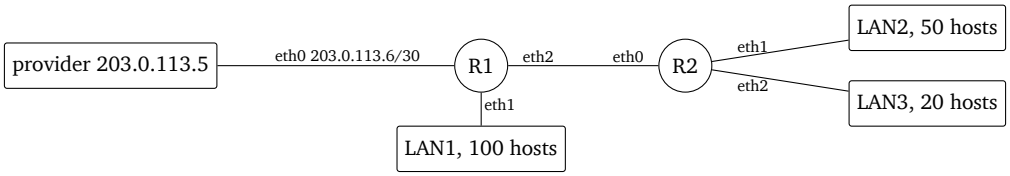
A browser sends an HTTP request; it goes down through TCP, IPv4 and Ethernet.

(a) Name the 7 OSI layers and the 4 TCP/IP layers and match them. Give the PDU name at the transport, network, datalink and physical layers. (1.5 pts)

(b) In which order do the headers appear in the transmitted frame? What is encapsulation, and why is the UDP checksum, which covers fields of the IP header, called a layering violation? (1 pt)

(c) Statistical multiplexing: $N = 400$ subscribers share a multiplexer; each is active with probability $p = 2\%$ at a given time, independently. What is the average number of active subscribers? Which distribution does the number X of active subscribers follow, and which approximation does the course use? With Chebyshev's inequality for $k = 3$ (take $\sigma = \sqrt{\lambda}$ for the Poisson distribution), which capacity L (an integer number of links) guarantees a blocking probability $\Pr(X > L)$ of at most 1/9? The exact Poisson computation gives $\Pr(X > L) \approx 0.16\%$ for that L: what do you conclude about Chebyshev? (2.5 pts)

2. Session 2: Interface addresses and routing tables (5 points) inspired by: 2018 midterm, Q7; 2019 practice set, ex. 1



The company owns the block 192.168.40.0/24 for LAN1, LAN2, LAN3 and the R1-R2 link.

(a) Split the block from its start, largest need first: network/prefix of LAN1, LAN2, LAN3 and of the link. (1.5 pts)

(b) Give an address to every interface of R1 and R2 on these networks, and the default gateway of a PC on LAN3. (1 pt)

(c) Write the routing tables of R1 and R2 (destination/prefix, next hop or interface), including the default route. (1.5 pts)

(d) Can R1 replace its routes to LAN2 and LAN3 with the single line 192.168.40.128/25? Follow a packet destined to 192.168.40.240. Why must R1 do NAT on eth0? (1 pt)

3. Session 3: DHCP, DNS and ICMP (5 points) inspired by: no past paper; hta3 p. 4 to 17; hta3p p. 18 to 20

A laptop with no network configuration joins a local network; the user then runs `traceroute www.example.org`.

(a) Describe the DHCP exchange: the four messages in order, which ones are broadcast and why, the transport and the ports, three pieces of information supplied besides the address. What happens if the lease is not renewed? (2 pts)

(b) How does the laptop obtain the IPv4 address of www.example.org: which server does it ask and how does it know that server, which record does it request, where can the answer come from? (1.5 pts)

(c) Explain how traceroute discovers the routers on the path: IP header field used, ICMP messages involved. (1.5 pts)

4. Session 4: One Bellman-Ford iteration (distance vector) (5 points) inspired by: 2017 topic list, no. 15; hta4p p. 3 and 4

Vector from	A	B	C	D	E	F
A (cost 2)	0	4	6	5	8	10
B (cost 7)	4	0	2	1	3	9
C (cost 3)	6	2	0	7	2	4

Router X has three neighbours, A, B and C, with the link costs shown. It has just received their distance vectors (left, destinations other than X).

(a) Compute X's new table from these vectors only: for each destination A to F, distance and next hop, with the formula used. (2.5 pts)

(b) Which vector does X advertise next? With split horizon, what does X's advertisement to C contain? (1 pt)

(c) Compare distance vector and link state (information sent, recipients, algorithm, convergence), with one protocol of each family. Which protocol is used between autonomous systems, and of which type is it? (1.5 pts)

5. Session 5: TCP connection, fast retransmit and window size (5 points) inspired by: 2020 exams; hta5 p. 9, 10; hta5p p. 13, 16

(a) A client opens a TCP connection with initial sequence number $S_c = 7300$; the server picks $S_s = 1200$. Give the flags, Sequence and Acknowledgement numbers of the three handshake segments. Which attack targets the server while it waits for the last segment, and which defense does the course give? (1.5 pts)

(b) As on the slides, numbers count segments. The sender sends segments 40 to 45; segment 42 is lost; the receiver acknowledges every segment it receives. List the ACKs. When does Reno retransmit, which ACK follows the arrival of the retransmitted 42, and what does Reno do to cwnd? (1.5 pts)

(c) A 200 Mbit/s link has a 60 ms RTT. Which window, in octets, is needed to use it fully? Which maximum throughput does the Window Size field alone allow (16 bits, counted in octets)? Which minimum window scale shift is needed? (2 pts)

6. Session 6: E-mail (5 points) inspired by: no past paper; hta6p p. 4 to 18

alice@example.org sends, from her mail client, a message with a PDF attachment to bob@example.net, who reads his mail on his phone and on his computer.

(a) Describe the path of the message: agents crossed (MUA, MTA, MDA), protocol and port at each step, and how the MTA of example.org finds the one of example.net. (2 pts)

(b) Why should Bob use IMAP rather than POP3? (1 pt)

(c) The message format only allows 7-bit ASCII: how is the attachment carried? (1 pt)

(d) How can the MTA of example.net check that the message comes from a server authorized for example.org? Name two mechanisms and their principle. (1 pt)

Solutions

Solutions and indicative marking schemes written by us, 5 points per question. According to the lecturer's remarks reported by Elisha, marking is lenient: an approximate explanation that goes in the right direction earns points. All calculations are checked by `calc_examens_blancs_net.py`. For each question, the filled boxes give the final answer to write on the paper; they are followed by the Method (general steps for any question of that type), the Formulas (with the slide where they appear; anything else is tagged [not in 2026 slides]), the Worked solution with every intermediate step, a Check, and the Marking and traps.

Paper A: solutions

6 questions, 5 points each, 30 points, pass mark 12/30. Justify every answer.

1. IPv4 addressing (5 points)

inspired by: 2018 midterm, "same subnet"; hta2p p. 12

A host is configured as 172.20.83.140/21.

(a) Give the netmask in dotted decimal notation. (1 pt)

Final answer

/21 = 8 + 8 + 5: 255.255.**248**.0 (248 = 11111000).

(b) Compute the network address, the broadcast address and the number of assignable host addresses. (2 pts)

Final answer

The interesting octet is the 3rd; block size $m = 256 - 248 = 8$. 83 = 01010011; the largest multiple of 8 not above 83 is 80. Network **172.20.80.0/21**, broadcast **172.20.87.255**, $2^{11} - 2 =$ **2046** hosts (172.20.80.1 to 172.20.87.254).

(c) For each address below, say whether it is valid, whether it is on the same subnet as the host, and whether it can be assigned to another host of that subnet: 172.20.86.17; 172.20.79.254; 172.20.88.1; 172.20.87.255; 172.20.80.0; 172.20.84.300. (2 pts)

Final answer

Address	Valid	Same subnet	Assignable
172.20.86.17	yes	yes (86 is in 80 to 87)	yes
172.20.79.254	yes	no (network 172.20.72.0/21)	no
172.20.88.1	yes	no (network 172.20.88.0/21)	no
172.20.87.255	yes	yes	no: broadcast address
172.20.80.0	yes	yes	no: network address
172.20.84.300	no: 300 > 255		

Method.

1. Split the prefix length n into octets: $n = 8 + 8 + \cdots + r$; the mask has as many 255 octets as full 8s, then one octet with r ones on the left, then zeros.
2. Write the partial octet of the address in binary (8 bits).
3. Network address: keep the n prefix bits, set the host bits to 0. Broadcast: set the host bits to 1.
4. Number of hosts: $2^{32-n} - 2$.
5. Same subnet: the first n bits are equal (equivalently, the same network address). Assignable: valid octets, same subnet, neither the network nor the broadcast address.

Formulas.

- Netmask of /n: n ones followed by $32 - n$ zeros (hta2p p. 6 and 11).
- Hosts of a /n: $2^{32-n} - 2$; the host ID cannot be all 0s (network) or all 1s (broadcast) (hta2p p. 4 and 7).
- Same subnet $\iff$ same first n bits (hta2p p. 12).
- Network = address AND mask; block size $m = 256-$ partial mask octet [not in 2026 slides, shortcuts for the bit comparison].

Worked solution. (a) 21 = 8 + 8 + 5: 255.255.11111000.0 = 255.255.248.0. (b) 3rd octet: 83 = 64 + 16 + 2 + 1 = 01010011.

Address, 3rd octet	01010 011 (83)
Mask, 3rd octet	11111 000 (248)
AND (network)	01010 000 = 80
Host bits set to 1 (broadcast)	01010 111 = 87

Network 172.20.80.0, broadcast 172.20.87.255 (4th octet all ones: 255). Host bits: $32 - 21 = 11$, $2^{11} - 2 = 2048 - 2 = 2046$; first host 172.20.80.1, last 172.20.87.254. (c) Prefix bits of the 3rd octet: 01010. 86 = 01010110: 01010, same. 79 = 01001111: 01001, different; its network is 01001000 = 72. 88 = 01011000: 01011, different; network 88. 172.20.87.255: host bits all 1, broadcast. 172.20.80.0: host bits all 0, network. 300 does not fit in 8 bits.

Check. $m = 256 - 248 = 8$; 80 is the largest multiple of 8 not above 83, and $80 + 8 - 1 = 87$. $8 \times 256 = 2048$ addresses, minus 2.

Marking and traps. (a) 1. (b) network 1, broadcast and host count 1. (c) 2, minus 0.5 per wrong row. **Traps.** Forgetting the -2; assuming 172.20.79.254 is a neighbour because it is numerically close; calling the broadcast and network addresses assignable because they are on the subnet.

2. Longest prefix match (5 points)

inspired by: 2018 midterm, routing table; hta2p p. 13 and 14

Router R uses the routing table on the left.

Prefix	Interface
84.112.0.0/12	eth0
84.120.0.0/13	eth1
84.124.0.0/14	eth2
84.126.64.0/18	eth3
0.0.0.0/0	eth4

(a) Give the outgoing interface for packets destined to 84.127.70.5, 84.126.100.1, 84.119.3.3 and 84.128.0.1. For each one, list the matching lines. (4 pts)

Final answer

Ranges: /12: 84.112.0.0 to 84.127.255.255; /13: 84.120 to 84.127; /14: 84.124 to 84.127; /18: 84.126.64.0 to 84.126.127.255.

Destination	Matching lines	Interface
84.127.70.5	/12, /13, /14, /0 (not the /18: 2nd octet is 127, not 126)	**eth2**
84.126.100.1	/12, /13, /14, /18 (100 is in 64 to 127), /0	**eth3**
84.119.3.3	/12, /0 (119 < 120, outside the /13)	**eth0**
84.128.0.1	/0 only	**eth4**

(b) An administrator wants to add the line 84.126.100.0/18 towards eth5. Why is it written incorrectly, and which network does it actually denote? (1 pt)

Final answer

In a /18 the 3rd octet of the network is a multiple of 64, and 100 is not. The line actually denotes 84.126.64.0/18 (100 = 01100100, keep 01), which already exists towards eth3: conflict.

Method.

1. Write the octets that the prefixes cut in binary and mark the prefix bits of each line.
2. For each destination, keep the lines whose prefix bits equal the destination's first bits.
3. Among those, the longest prefix wins; the default route matches everything and is used only when nothing longer matches.
4. A line is well formed only if all its host bits are 0.

Formulas.

- Longest prefix match wins (hta2p p. 13).
- A $/n$ covers 2^{32-n} addresses from the network address (hta2p p. 7).
- 0.0.0.0/0 as default route [not in 2026 slides; they speak of the configured gateway, hta2 p. 6].

Worked solution. 2nd octet: /12 keeps 4 bits, /13 keeps 5, /14 keeps 6, /18 keeps the whole 2nd octet and 2 bits of the 3rd.

Line	Bits	Range
84.112.0.0/12	2nd: 0111 0000 (112)	84.112 to 84.127
84.120.0.0/13	2nd: 01111 000 (120)	84.120 to 84.127
84.124.0.0/14	2nd: 01111 00 (124)	84.124 to 84.127
84.126.64.0/18	2nd: 01111110, 3rd: 01 000000 (64)	84.126.64 to 84.126.127

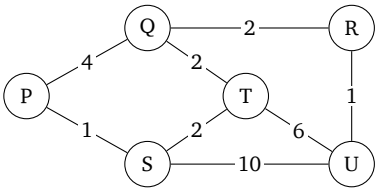
84.127.70.5: 127 = 01111111: matches 0111, 01111, 011111; not 01111110: longest /14, eth2. 84.126.100.1: 126 = 01111110, 100 = 01100100 starts with 01: /18 matches, eth3. 84.119.3.3: 119 = 01110111: 0111 yes, 01111 no: eth0. 84.128.0.1: 128 = 10000000: no line, default eth4. (b) /18: 2 prefix bits in the 3rd octet; 100 = 01 100100: the 6 host bits are not 0. Keeping 01: 01000000 = 64, so 84.126.64.0/18.

Check. Range sizes: $2^{32-12} = 2^{20}$, 16 values of the 2nd octet (112 to 127); /13: 8 values (120 to 127); /14: 4 values (124 to 127); /18: 64 values of the 3rd octet (64 to 127).

Marking and traps. (a) 1 per destination, 0.5 of it for the justification. (b) 1. **Traps.** 84.127.70.5 has its 3rd octet in 64 to 127 but a different 2nd octet; the default route is used only when nothing else matches.

3. Link state: Dijkstra's algorithm (5 points)

inspired by: 2017 topic list, no. 16; hta4p p. 8 to 14



Link costs are shown on the graph.

(a) Run Dijkstra's algorithm from P. Give a table: at each step, the node taken from the priority queue and, for every other node, the tentative distance and the previous node. On a tie, take the smallest letter. (3 pts)

Final answer

Cell "d,p": tentative distance, previous node; bold: the node taken at that step (final).

Step	Taken	Q	R	S	T	U
0	P (0)	4,P	∞	1,P	∞	∞
1	S (1)	4,P	∞	**1**	3,S	11,S
2	T (3)	4,P	∞		**3**	9,T
3	Q (4)	**4**	6,Q			9,T
4	R (6)		**6**			7,R
5	U (7)					**7**

At step 2 the path P-S-T-Q costs 5, not better than 4: Q keeps P as previous node.

(b) Derive the FIB of P: destination, cost, next hop. (1 pt)

Final answer

Q 4 via Q; R 6 via Q (P-Q-R); S 1 via S; T 3 via S (P-S-T); U 7 via Q (P-Q-R-U).

(c) The Q-R link fails. Give the new costs from P to R and to U. According to the course, which family reconfigures faster after a link failure: link state or distance vector? (1 pt)

Final answer

Without Q-R: U = 9 via P-S-T-U, R = 10 via P-S-T-U-R, both via S. The slides state that LS can detect and reconfigure faster than DV after a link failure.

Method.

1. Start: $N' = \{\text{source}\}$; $D(v)$ = link cost for the source's neighbours, ∞ otherwise; $p(v)$ = source for the neighbours.
2. Take the node $w \notin N'$ with the smallest $D(w)$ (ties: smallest letter), add it to N': its distance is final.
3. For each neighbour $v \notin N'$ of w: if $D(w) + c(w,v) < D(v)$, set $D(v) = D(w) + c(w,v)$ and $p(v) = w$.
4. Repeat until every node is in N'. The FIB keeps, for each destination, the first hop found by following $p(v)$ back to the source.

Formulas.

- Relaxation: $d = u.\text{dist} + \text{dist}(u,v)$; if $d < v.\text{dist}$ then update (hta4p p. 3).
- Equivalently $D(v) \leftarrow \min\big(D(v), D(w) + c(w,v)\big)$ [written form of the slide's pseudo-code].

Worked solution.

Step	N'	$D(Q), p$	$D(R), p$	$D(S), p$	$D(T), p$	$D(U), p$
0	P	4,P	∞	1,P	∞	∞
1	P S	4,P	∞		$1+2=3$,S	$1+10=11$,S
2	P S T	$3+2=5 \not< 4$: 4,P	∞			$3+6=9<11$: 9,T
3	P S T Q		$4+2=6$,Q			9,T
4	P S T Q R					$6+1=7<9$: 7,R
5	all					

First hops: Q: $p(Q) = $ P, so Q. R: R → Q → P, so Q. S: S. T: T → S, so S. U: U → R → Q → P, so Q. (c) Without Q-R, R is only reachable through U: $D(U) = \min(1 + 10,\ 3 + 6) = 9$ via S-T, $D(R) = 9 + 1 = 10$.

Check. Each final $D(v)$ is the minimum over its neighbours of D(neighbour)+cost: $D(U) = \min(6+1,\ 3+6,\ 1+10) = 7$; $D(Q) = \min(0+4,\ 3+2,\ 6+2) = 4$.

Marking and traps. (a) 3 (2 if one isolated error is propagated consistently). (b) 1. (c) 0.5 costs, 0.5 comparison. **Traps.** Taking the direct S-U (10) or T-U (6) link for U; updating a node that has already been taken; writing the whole path in the FIB instead of the next hop.

4. TCP Reno congestion control (5 points)

inspired by: 2017 topic list, no. 33; hta5p p. 8 to 15

Conventions: cwnd and ssthresh in MSS, one round per round trip, cwnd segments sent per round. Slow Start (SS): cwnd doubles every round, **capped at ssthresh** (if doubling would exceed ssthresh, cwnd takes the value ssthresh), then Congestion Avoidance (CA). CA: cwnd +1 per round and ssthresh = cwnd. Three duplicate ACKs: cwnd and ssthresh both become half of cwnd, stay in CA. Timeout: ssthresh = cwnd/2, cwnd = 1, back to SS. A loss detected at the end of a round changes the next round; adv_wnd never limits the sender. In round 1, cwnd = 1 and ssthresh = 12. The sender receives three duplicate ACKs at the end of round 9 and has a timeout at the end of round 14.

(a) Give cwnd, ssthresh and the phase (SS or CA) for rounds 1 to 19. (3 pts)

Final answer

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
cwnd	1	2	4	8	12	13	14	15	16	8	9	10	11	12	1	2	4	6	7
ssthresh	12	12	12	12	12	13	14	15	16	8	9	10	11	12	6	6	6	6	7
Phase	SS	SS	SS	SS	CA	CA	CA	CA	CA	CA	CA	CA	CA	CA	SS	SS	SS	CA	CA

Round 5: doubling would give 16, capped at 12. End of round 9: 3 duplicate ACKs with cwnd = 16, so cwnd = ssthresh = 8. End of round 14: timeout with cwnd = 12, so ssthresh = 6, cwnd = 1; round 18: 8 capped at 6. Rounds 5 and 18, where cwnd reaches ssthresh, may be marked SS or CA: what matters is that the next round grows by +1.

(b) How many segments are sent in total during rounds 1 to 19? (1 pt)

Final answer

$1 + 2 + 4 + 8 + 12 + 13 + 14 + 15 + 16 = 85$ in rounds 1 to 9; **155** segments in total.

(c) Why does Reno wait for three duplicate ACKs rather than one, and why does it treat a timeout more severely? (1 pt)

Final answer

Out-of-order delivery without loss also produces duplicate ACKs; three of them make a loss very likely. Duplicate ACKs mean that segments still get through (mild congestion: halve cwnd). A timeout means nothing gets through: the congestion looks severe, back to slow start.

Method.

1. One line per round: write cwnd, ssthresh and the phase used in that round.
2. Next round: slow start doubles cwnd (capped at ssthresh by the stated convention); congestion avoidance adds 1 and sets ssthresh = cwnd.
3. On a loss at the end of a round, apply the loss rule to get the next round's values.
4. Segments sent in a round = cwnd; total = sum of the cwnd values.

Formulas.

- Slow start: start cwnd = 1, cwnd ×2 per round until cwnd reaches ssthresh (hta5p p. 8); the cap at ssthresh is the convention of the question [not in 2026 slides].
- Congestion avoidance: cwnd +1 per round and ssthresh = cwnd (hta5p p. 9).
- Timeout: ssthresh/2 and back to slow start with cwnd = 1 (hta5p p. 8, 9); since ssthresh = cwnd in congestion avoidance, the new ssthresh is cwnd/2.
- Three duplicate ACKs (Reno fast recovery): cwnd/2, stay in congestion avoidance, ssthresh keeps following cwnd (hta5p p. 14).
- Data sent = area under the cwnd curve (hta5p p. 15).

Worked solution. Rounds 1 to 4: 1 → 2 → 4 → 8. Round 5: $2 \times 8 = 16 > 12$, capped: 12 = ssthresh, congestion avoidance from now on. Rounds 6 to 9: 13, 14, 15, 16 (ssthresh follows). End of round 9, 3 duplicate ACKs: 16/2 = 8, cwnd = ssthresh = 8. Rounds 10 to 14: 8, 9, 10, 11, 12. End of round 14, timeout: ssthresh = 12/2 = 6, cwnd = 1. Rounds 15 to 17: 1, 2, 4. Round 18: $2 \times 4 = 8 > 6$, capped at 6, congestion avoidance. Round 19: 7. (b) $1 + 2 + 4 + 8 + 12 = 27$; $27 + 13 + 14 + 15 + 16 = 85$; $85 + 8 + 9 + 10 + 11 + 12 = 135$; $135 + 1 + 2 + 4 + 6 + 7 = 155$.

Check. The two loss reactions differ as on the slides: after duplicate ACKs cwnd goes from 16 to 8 and keeps growing by 1; after the timeout it restarts at 1. 19 values listed.

Marking and traps. (a) 3: 1 for slow start with the cap, 1 for fast recovery, 1 for the timeout. (b) 1. (c) 1. **Traps.** Doubling to 16 in round 5 without applying the convention; going back to 1 after duplicate ACKs (that is Tahoe); halving the initial ssthresh (12) instead of cwnd at the time of the loss.

5. Serialization: XML to JSON and YAML (5 points)

inspired by: hta1p p. 26, "Possible exam question"

A DHCP server exports a lease in the XML format shown on the left.

```
<lease>
  <ip>192.168.10.57</ip>
  <hostname>labo-pc7</hostname>
  <duration>86400</duration>
  <options>
    <router>192.168.10.1</router>
    <dns>192.168.10.2</dns>
    <dns>192.168.10.3</dns>
  </options>
</lease>
```

(a) Write the same content in JSON. (2 pts)

Final answer

Slide convention (hta1p p. 24 to 26): the root element disappears and its content becomes the top-level object; repeated siblings become an array. Keeping the root as a key (`"lease": {…}`) is also consistent; we accept it.

```
{
  "ip": "192.168.10.57",
  "hostname": "labo-pc7",
  "duration": 86400,
  "options": {
    "router": "192.168.10.1",
    "dns": ["192.168.10.2", "192.168.10.3"]
```

```
  }
}
```

(b) Write it in YAML. (2 pts)

Final answer

```
ip: 192.168.10.57
hostname: labo-pc7
duration: 86400
options:
  router: 192.168.10.1
  dns:
    - 192.168.10.2
    - 192.168.10.3
```

(c) Why does the course recommend `JSON.parse()` rather than `eval()` to read JSON in JavaScript? (1 pt)

Final answer

`eval()` runs the text as JavaScript code: a malicious text would execute whatever it contains. `JSON.parse()` only reads data, so it is more secure.

Method.

1. The root element disappears: its children become the keys of the top-level object.
2. An element with text becomes `key: value`; in JSON a number stays unquoted, a string is quoted (an IP address is a string).
3. An element with different children becomes a nested object.
4. Repeated siblings with the same name become one key holding an array (the repeated tag is not written again).
5. YAML: the same tree, `key: value` per line, two more spaces per level, one hyphen per list item, no braces.

Formulas. No formula; the rules are those of the slide example (hta1p p. 24 to 26: the `NTALecture` record in XML, JSON and YAML).

Worked solution. `<ip>`, `<hostname>`: strings → quoted. `<duration>` 86400: number → unquoted. `<options>` has two different child names → nested object; inside it `<router>` is a single string; the two `<dns>` siblings → `"dns": [ …]`. YAML: `options:` on its own line, then `router` and `dns` indented by two spaces, the two addresses as `-` items indented once more.

Check. The XML holds 6 text values (ip, hostname, duration, router, 2 dns); the JSON and the YAML each hold the same 6 values, none lost or duplicated.

Marking and traps. (a) 2: structure 1, `dns` array 0.5, number without quotes and addresses quoted 0.5. (b) 2: indentation 1, hyphenated list 1. (c) 1. **Traps.** Two `"dns"` keys in the same object (the second overwrites the first); quoting 86400; losing the indentation under `options`.

6. Multiple access, duplexing and token bucket (5 points)

inspired by: 2018 midterm, Q1; hta1 p. 19 to 21; hta1p p. 18 and 19

A radio operator has two 1.6 MHz bands, one for the uplink and one for the downlink. Each band is split into 200 kHz carriers, and each carrier into frames of 8 time slots. A call uses one slot on an uplink carrier and one slot on a downlink carrier.

(a) Define FDMA, TDMA and CDMA in one sentence each. Which multiple access and duplexing techniques does this operator combine? What is the maximum number of simultaneous calls? (2 pts)

Final answer

FDMA: each source mixes with a different carrier. TDMA: each source has a time slot for sending, which needs carrier sense or time synchronization. CDMA: each source uses a different spreading code, on the same band at the same time; the codes cannot be fully orthogonal if the sources are not perfectly synchronized. The operator combines FDMA and TDMA (carriers split into slots) with Frequency Division Duplex (FDD: one band per direction). 1.6 MHz/200 kHz = 8 carriers per direction, ×8 slots: **64** calls.

(b) A second operator has a single 3.2 MHz band, with the same carriers and frames, and uses Time Division Duplex (TDD). What is the principle of TDD? What is the maximum number of symmetric calls? Which constraint does time sharing impose on the equipment? (1 pt)

Final answer

TDD: both directions use the same frequency and take turns. 3.2 MHz/200 kHz = 16 carriers ×8 = 128 slots; a call needs two (one per direction): **64** calls, as many as with FDD and the same total bandwidth. Constraint: the equipment must be time synchronized to know when to send and when to receive.

(c) At the network entrance, a client's flow is metered by a token bucket (one token = one bit). Contract: average bitrate 4 Mbit/s, maximum bitrate 10 Mbit/s, bursts of 0.3 s. Give K and B with the design rule of the course. The bucket is full and the client sends at 10 Mbit/s: after how long is the bucket empty, and what happens then to the traffic above 4 Mbit/s? What would a leaky bucket do instead? (2 pts)

Final answer

K = average bitrate = 4 Mbit/s ($4 \cdot 10^6$ tokens/s). $B = (\text{max} - \text{avg}) \times \text{burstlen} = (10 - 4) \times 0.3 =$ **1.8 Mbit**. The bucket loses $10 - 4 = 6$ Mbit/s net, so it is empty after $1.8/6 =$ **0.3 s**, the contracted burst length. Then only the messages covered by new tokens pass (4 Mbit/s); the excess is dropped. A leaky bucket shapes the flow: output at most K, the excess waits in a queue and is dropped only when the bucket is full.

Method.

1. Separate who shares the medium (multiple access: frequency, time, code) from how the two directions are separated (duplexing: FDD, TDD, separate wires).
2. Count channels: carriers per band, then slots per carrier; one call needs one channel per direction.
3. Token bucket: set K and B from the contract, then compare the input rate with K.

Formulas.

- FDMA, TDMA, CDMA, FDD, TDD definitions (hta1 p. 19 and 20).
- Carriers = band width / carrier width; channels = carriers × slots [not in 2026 slides, simple counting].
- Token bucket design: K = average bitrate, $B = (\text{max} - \text{avg}) \times \text{burstlen}$ (hta1p p. 18 and 19).
- Time to empty a full bucket: $B/(\text{input rate} - K)$ [not in 2026 slides].

Worked solution. (a) 1.6 MHz = 1600 kHz; 1600/200 = 8 carriers per direction; $8 \times 8 = 64$ uplink slots and 64 downlink slots; one of each per call: 64 calls. (b) 3200/200 = 16 carriers; $16 \times 8 = 128$ slots shared by both directions; 128/2 = 64 calls. (c) $K = 4$ Mbit/s; $B = (10 - 4) \times 0.3 = 6 \times 0.3 = 1.8$ Mbit. Tokens leave at 10 and arrive at 4 Mbit/s: net $10 - 4 = 6$ Mbit/s; $1.8/6 = 0.3$ s.

Check. In 0.3 s the client sends $10 \times 0.3 = 3$ Mbit $= B + K \times 0.3 = 1.8 + 1.2$. FDD and TDD give the same number of calls for the same total bandwidth.
Marking and traps. (a) 2: definitions 1, FDMA, TDMA and FDD named 0.5, 64 calls 0.5. (b) 1: principle and synchronization 0.5, 64 calls 0.5. (c) 2: K and B 1, 0.3 s and fate of the excess 0.5, leaky bucket 0.5. **Traps.** Answering that FDMA and TDD cannot work together (they combine); stopping at the definitions without describing the combination; counting 128 calls in TDD and forgetting one slot per direction; mixing up metering (token bucket, drop) and shaping (leaky bucket, queue).

1. Variable length subnetting (5 points) inspired by: 2019 practice set, ex. 6 (splitting a /20); hta2p p. 7

A company owns the block 10.62.96.0/20. Needs: Production 1000 hosts, Offices 450, Laboratory 120, Guest Wi-Fi 60, and two point-to-point links between routers (two addresses each).

(a) Which prefix length does each need require? (1 pt)

Final answer

Smallest h with $2^h - 2 \geq$ need: 1000 gives /22 (1022); 450 gives /23 (510); 120 gives /25 (126); 60 gives /26 (62); links /30 (2).

(b) Allocate the subnets from the start of the block, largest first. For each one, give network/prefix and broadcast address. (3 pts)

Final answer

Need	Network/prefix	Broadcast	Hosts
Production	10.62.96.0/22	10.62.99.255	1022
Offices	10.62.100.0/23	10.62.101.255	510
Laboratory	10.62.102.0/25	10.62.102.127	126
Guest Wi-Fi	10.62.102.128/26	10.62.102.191	62
Link 1	10.62.102.192/30	10.62.102.195	2
Link 2	10.62.102.196/30	10.62.102.199	2

Each block starts on a multiple of its size (100 is even for the /23, 102.128 is a multiple of 64): allocating in decreasing order guarantees alignment.

(c) What is the largest aligned subnet still free in the block? Which single prefix does the company announce to its provider? (1 pt)

Final answer

Free: 10.62.102.200 to 10.62.111.255, 2360 addresses. Largest aligned block: **10.62.104.0/21** (104 to 111). Single announcement: **10.62.96.0/20**.

Method.

1. For each need, find the smallest h with $2^h - 2 \geq$ hosts; prefix $= 32 - h$.
2. Sort the needs from largest to smallest.
3. Allocate from the start of the block: each subnet starts at the first free address, which is then aligned on its size; broadcast = network $+2^h - 1$.
4. Free space = block size minus the allocated sizes; look for the largest block whose start is a multiple of its size and which is completely free.

Formulas.

- Hosts of a /n: $2^{32-n} - 2$ (hta2p p. 7); subnetting a block into smaller networks (hta2p p. 5).
- Largest-first allocation (VLSM) [not in 2026 slides].

Worked solution. 1000: $2^9 - 2 = 510 < 1000$, $2^{10} - 2 = 1022 \geq 1000$: $h = 10$, /22. 450: $2^8 - 2 = 254 < 450$, $2^9 - 2 = 510$: /23. 120: $2^7 - 2 = 126$: /25. 60: $2^6 - 2 = 62$: /26. Links: $2^2 - 2 = 2$: /30.

Need	Size	Start	End	Next free
Production /22	$1024 = 4 \times 256$	10.62.96.0	10.62.99.255	10.62.100.0
Offices /23	$512 = 2 \times 256$	10.62.100.0	10.62.101.255	10.62.102.0
Laboratory /25	128	10.62.102.0	10.62.102.127	10.62.102.128
Guest Wi-Fi /26	64	10.62.102.128	10.62.102.191	10.62.102.192
Link 1 /30	4	10.62.102.192	10.62.102.195	10.62.102.196
Link 2 /30	4	10.62.102.196	10.62.102.199	10.62.102.200

(c) The /20 has $2^{12} = 4096$ addresses, 10.62.96.0 to 10.62.111.255 ($96+16-1 = 111$). Used: $1024+512+128+64+4+4 = 1736$; free: $4096-1736 = 2360$, from 10.62.102.200 to 10.62.111.255. The two /21 halves are 96 to 103 and 104 to 111; the second is entirely free: 10.62.104.0/21 (2048 addresses).

Check. $1736 + 2360 = 4096$. Each start is a multiple of its size: 96 is a multiple of 4 (a /22 spans 4 values of the 3rd octet), 100 is even (a /23 spans 2), 128 is a multiple of 64, 192 and 196 are multiples of 4.

Marking and traps. (a) 1. (b) 3, 0.5 per row. (c) 0.5 and 0.5. **Traps.** /23 for 1000 hosts ($2^9 - 2 = 510$); /27 for the guests ($2^5 - 2 = 30$); starting with the small blocks, which breaks the alignment of the large ones.

2. CIDR aggregation (5 points) inspired by: 2018 midterm, Q10; hta2p p. 14

Prefix	Next hop
185.40.16.0/22	A
185.40.20.0/22	A
185.40.24.0/21	A
185.40.32.0/21	A
185.40.40.0/22	B
185.40.44.0/22	A
185.40.48.0/20	A

A router has the table on the left and no default route.

(a) Give the address range of each line (the 3rd octet is enough). (1 pt)

Final answer

3rd octet: 16 to 19; 20 to 23; 24 to 31; 32 to 39; 40 to 43 (B); 44 to 47; 48 to 63.

(b) Reduce the table to the minimum number of lines by exact merging: the new table covers exactly the same addresses, every address keeps its next hop, and no line may overlap another. (2.5 pts)

Final answer

16/22 + 20/22: contiguous, same size, 16 is a multiple of 8, so 16.0/21; + 24/21: 16 is a multiple of 16, so **185.40.16.0/20**. 32/21 could only merge with 40/21, half of which (40 to 43) goes to B. 44/22 could only merge with 40/22 (B). 16/20 and 32/21 differ in size, and 16/20 would only merge with 0/20 (absent). Result, 5 lines: 185.40.16.0/20 A; 185.40.32.0/21 A; 185.40.40.0/22 B; 185.40.44.0/22 A; 185.40.48.0/20 A.

(c) Now a more specific line may override a wider one (longest prefix match). Reduce the table to three lines without changing the output of any address. Why can't everything be summarized as 185.40.0.0/18 towards A? (1.5 pts)

Final answer

185.40.16.0/20 A; **185.40.32.0/19** A (32 to 63); 185.40.40.0/22 B. The /22 towards B is more specific and wins over the /19 for 40 to 43; every other address keeps A. 185.40.0.0/18 would also cover 185.40.0.0 to 185.40.15.255, which are not in the table: these packets, dropped today for lack of a route, would go to A.

Method.

1. Write each line's range and the bits it fixes.
2. Merge two lines only if they have the same next hop and the same size, are contiguous, and only the last bit of their prefixes differs (the merged block is then aligned).
3. Repeat with the merged blocks; check that the result covers exactly the same addresses.
4. With longest prefix match, a wider line may cover a more specific exception.

Formulas.

- Merge rule: same first $n-1$ bits and same next hop gives one $/(n-1)$ (hta2p p. 6); "we can merge if only the last bit of the prefix differs" (hta2p p. 14).
- Size in the 3rd octet: /22 → 4, /21 → 8, /20 → 16, /19 → 32 values (2^{24-n}) (hta2p p. 7).
- Longest prefix match (hta2p p. 13).

Worked solution. 3rd octets: 16 = 00010000, 20 = 00010100, 24 = 00011000, 32 = 00100000, 40 = 00101000, 44 = 00101100, 48 = 00110000. A /22 fixes 6 bits of the 3rd octet, a /21 five, a /20 four.

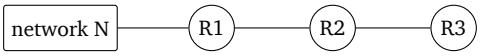
16/22 000100 and 20/22 000101: only the last bit differs → 00010 = 16/21. 16/21 00010 and 24/21 00011 → 0001 = 16/20. 32/21 00100: its partner 00101 (40/21) is half B, no. 44/22 001011: partner 001010 is 40/22 (B), no. 16/20 0001: partner 0000 (0/20) absent. 48/20 0011: partner 0010 (32/20) mixes A and B, no. (c) 32/19 001 covers 32 to 63, with 40/22 (B) more specific. 0/18 00 would add 0 to 15.

Check. Values of the 3rd octet sent to A: before $4+4+8+8+4+16=44$; after (b) $16+8+4+16=44$; after (c) $16+32-4=44$. B keeps its 4 values.

Marking and traps. (a) 1. (b) 2.5: 16.0/20 1.5, other merges correctly refused 1. (c) 1.5: three lines 1, /18 refused 0.5. **Traps.** Merging 32/21 and 40/22 although 40/22 goes to B; writing 185.40.16.0/19 (16 is not a multiple of 32); aggregating wider than the addresses present.

3. Distance vector: count to infinity (5 points)

inspired by: 2018 midterm, Q3; 2020 exams; hta4p p. 4 to 6



The routers run a RIP-like distance vector protocol: hop count metric, R1 is directly connected to N with distance 1, infinity is 16.

(a) After convergence, give the distance and next hop towards N in R1, R2 and R3. What does R2's advertisement contain about N, and what is not in it? (1.5 pts)

Final answer

R1: 1, directly connected; R2: 2 via R1; R3: 3 via R2. R2 advertises (N, 2): destination and distance, without the next hop. So R3 does not know that R2's route goes through R1.

(b) The R1-R2 link goes down. R2 marks N unreachable, but receives R3's periodic update before it has advertised the failure; updates then alternate, R3 then R2. Describe the first four updates (router, distance, next hop). After how many updates do both R2 and R3 consider N unreachable? What happens meanwhile to packets towards N? (2 pts)

Final answer

1. R2 receives "N at 3" from R3 and sets N to 4 via R3 (it does not know this route goes through itself). 2. R3 receives "N at 4" from its next hop R2 and moves to 5 via R2: a router always follows its next hop, even if the distance grows. 3. R2 moves to 6 via R3. 4. R3 moves to 7 via R2. After k updates the distance is $k+3$: R2 reaches 16 at the 13th, R3 at the **14th**. Meanwhile, packets towards N bounce between R2 and R3 until their TTL expires.

(c) Explain split horizon and poisoned reverse on this example: what does R3 advertise to R2 about N in each case, and what becomes of scenario (b)? (1.5 pts)

Final answer

Split horizon: R3 does not advertise N to R2, its next hop for N; R2 keeps N unreachable and advertises that to R3, which marks it unreachable in turn. Poisoned reverse: R3 advertises N with an infinite cost (16) to R2, which will not choose R3 as next hop. Either way, no counting. Without a fix, RIP's hop count limit of 15 at least bounds the counting.

Method.

1. Converged state: on a line, the distance grows by 1 per hop from the destination.
2. After a failure, a router that receives "destination at y" from a neighbour uses $y+1$ if that neighbour is its next hop or offers a shorter route.
3. Alternate the updates between the two routers and stop when both reach infinity (16).

Formulas.

- DV update: a neighbour reaching C in y hops gives $y+1$ hops through it (hta4p p. 4).
- RIP: hop count, limit 15, so 16 means unreachable (hta4 p. 10).
- Split horizon, poisoned reverse (hta4p p. 6).

Worked solution. (a) R1: 1; R2: $1+1=2$ via R1; R3: $2+1=3$ via R2. (b)

Update k	Sender	Receiver	Computation	New distance
1	R3 (3)	R2	$3+1$	4 via R3
2	R2 (4)	R3	$4+1$	5 via R2
3	R3 (5)	R2	$5+1$	6 via R3
4	R2 (6)	R3	$6+1$	7 via R2
k				$k+3$
13	R3 (15)	R2	$15+1$	16 (infinity)
14	R2 (16)	R3	$16+1$, capped	16

Check. $k+3=16$ gives $k=13$ for R2; R3 follows at the next update, 14.

Marking and traps. (a) 1.5. (b) 2: mechanism 1, numbers 0.5, looping packets 0.5. (c) 1.5. **Traps.** Forgetting that R3 accepts a larger distance because it comes from its next hop; saying that the advertisement carries the path (that is BGP, path vector).

4. NAT (5 points) inspired by: no past paper; hta5 p. 18 and 19

Home network 192.168.0.0/24; NAT gateway with internal address 192.168.0.1 and external address 81.183.20.9. Hosts 192.168.0.23 and 192.168.0.42 each open a TCP connection to server 145.236.10.5, port 443, both with source port 51000. The gateway picks external ports 40100 (for .23) and then 40101 (for .42).

(a) For the connection of host .23, give source and destination (address and port) on the four legs: host to gateway, gateway to server, server to gateway, gateway to host. (2 pts)

Final answer

Leg	Source	Destination
host to gateway	192.168.0.23:51000	145.236.10.5:443
gateway to server	81.183.20.9:40100	145.236.10.5:443
server to gateway	145.236.10.5:443	81.183.20.9:40100
gateway to host	145.236.10.5:443	192.168.0.23:51000

(b) Write the gateway's association table. Why can't the gateway keep port 51000 for both connections? Reason with the 5-tuple. (1.5 pts)

Final answer

External port 40100 ↔ 192.168.0.23:51000; 40101 ↔ 192.168.0.42:51000 (TCP, towards 145.236.10.5:443). With 51000 for both, the two connections would have the same 5-tuple on the Internet side, {81.183.20.9, 145.236.10.5, TCP, 51000, 443}: the gateway could not tell which host a reply belongs to. A distinct external port per connection makes each 5-tuple unique.

(c) An internal web server, 192.168.0.50 port 8080, must be reachable from the Internet. Why doesn't the NAT described here allow it, and which solutions does the course mention? (1.5 pts)

Final answer

Source NAT (S-NAT) only works for connections initiated from the private network: a SYN arriving at 81.183.20.9:8080 matches no association and cannot be forwarded. Solutions: D-NAT, a fixed rule mapping an external port to 192.168.0.50:8080; or NAT traversal, where the application asks the gateway to open a port for incoming connections (UPnP Internet Gateway Protocol).

Method.

1. Outbound: replace the source address and port with the external address and a new port; remember the association.
2. Inbound: look up the destination port in the table, write back the original address and port.
3. Every connection needs its own external port so that the outside 5-tuples are distinct.

Formulas.

- Conversation identified by the 5-tuple {IP 1, IP 2, protocol, port 1, port 2} (hta5 p. 4).
- NAT replaces source IP and source port, remembers the association (hta5 p. 18).

Worked solution. Leg 1: unchanged packet from the host. Leg 2: source 192.168.0.23:51000 → 81.183.20.9:40100, destination unchanged. Leg 3: the server answers to the source it saw, 81.183.20.9:40100. Leg 4: the gateway finds 40100 in its table and rewrites the destination to 192.168.0.23:51000. Second host: 192.168.0.42:51000 → 81.183.20.9:40101.

Check. The reply's 5-tuple is the outbound one with source and destination swapped: {145.236.10.5, 81.183.20.9, TCP, 443, 40100}; the two outside 5-tuples differ only by 40100 and 40101.

Marking and traps. (a) 2, 0.5 per leg. (b) 1.5: table 0.5, 5-tuple 1. (c) 1.5. **Traps.** Rewriting the destination instead of the source; forgetting the reverse rewriting on the way back.

5. Serialization: XML to JSON and YAML (5 points) inspired by: hta1p p. 26, "Possible exam question"

A router configuration is given in XML on the left.

```
<router>
  <name>edge1</name>
  <ospfArea>0.0.0.0</ospfArea>
  <interfaces>
    <interface><name>eth0</name>
      <address>10.0.0.1/30</address>
      <mtu>1500</mtu></interface>
    <interface><name>eth1</name>
      <address>192.168.5.1/24</address>
      <mtu>9000</mtu></interface>
  </interfaces>
  <protocols>
    <protocol>ospf</protocol>
    <protocol>bgp</protocol>
  </protocols>
</router>
```

(a) Write the same content in JSON. (2 pts)

Final answer

```
{
  "name": "edge1",
  "ospfArea": "0.0.0.0",
  "interfaces": [
    { "name": "eth0", "address": "10.0.0.1/30", "mtu": 1500 },
    { "name": "eth1", "address": "192.168.5.1/24", "mtu": 9000 }
  ],
  "protocols": ["ospf", "bgp"]
}
```

The repeated `<interface>` elements become an array of objects, the `<protocol>` elements an array of strings; the repeated tags disappear.

(b) Write it in YAML. (2 pts)

Final answer

```
name: edge1
ospfArea: 0.0.0.0
interfaces:
```

```
  - name: eth0
    address: 10.0.0.1/30
    mtu: 1500
  - name: eth1
    address: 192.168.5.1/24
    mtu: 9000
protocols:
  - ospf
  - bgp
```

(c) The course also presents TLV (Type, Length, Value) encoding. Why can a reader written for an older version of a TLV protocol still read a message that contains a new type? (1 pt)

Final answer

Every element carries its length: the reader skips the value of an unknown type and goes on.

Method.

1. Drop the root; its children become top-level keys.
2. Text elements: `key: value` (numbers unquoted in JSON, strings quoted).
3. A container whose children repeat the same tag becomes an array named after the container; if each repeated child has its own children, each array item is an object.
4. YAML: one hyphen per array item; the keys of the same item are aligned under the first one.

Formulas. No formula; rules of the slide example (hta1p p. 24 to 26). TLV: Type, Length, Value, the length lets the reader skip unknown parts (hta1p p. 22).

Worked solution. `name` and `ospfArea` are strings ("0.0.0.0" is an area ID, not a number). `<interfaces>` contains two `<interface>` children → `"interfaces": [ {…}, {…} ]`; each has `name` (string), `address` (string), `mtu` (number). `<protocols>` contains two `<protocol>` texts → `["ospf", "bgp"]`.

Check. 10 text values in the XML (2 scalars, 2×3 interface fields, 2 protocols), the same 10 in JSON and in YAML.

Marking and traps. (a) 2: array of objects 1, rest 1. (b) 2: hyphens and alignment of the keys of one element 1, rest 1. (c) 1. **Traps.** Writing `0.0.0.0` without quotes in JSON (it is not a number); in YAML, a hyphen before every key of an interface instead of one hyphen per list element.

6. Web: HTTP, TLS and QUIC; quality of service (5 points)

inspired by: no past paper; hta6 p. 8 to 15; hta1p p. 16

A browser opens `https://www.example.org/index.html` for the first time.

(a) List in order the exchanges needed until the page is received with HTTP/1.1: protocol, transport, port, role of each exchange. (2 pts)

Final answer

1. DNS: A record of www.example.org asked to the local resolver, UDP 53. 2. TCP: 3-way handshake (SYN; SYN,ACK; ACK) to port 443. 3. TLS: handshake after the TCP handshake; the server sends its certificate and a common encryption key is agreed. 4. HTTP: GET /index.html, reply 200 OK with the document, encrypted by TLS. With HTTP/1.1 the same TCP connection is then reused for the other objects of the page.

(b) The server presents an X.509 certificate. Under which two conditions does the browser accept it? What does a self-signed certificate allow, and what does it not allow? (1 pt)

Final answer

It was issued to the site we are connected to, and a CA that we trust has signed it. Self-signed: usable for encryption, not for validating the identity of the site.

(c) What changes with HTTP/3 over QUIC: transport, handshakes, encryption, mobility? (1 pt)

Final answer

QUIC runs over UDP, implemented in the application layer; the session and TLS handshakes are combined (3-way instead of 6-way); encryption is mandatory; the transport session ID is independent of the ports, which supports mobility; multiplexing is aware of web elements.

(d) Outside the Web, session 1: the course distinguishes IntServ, DiffServ and best effort. How do they differ? Which one describes how the Internet runs, and why? (1 pt)

Final answer

IntServ reserves resources for individual flows (fine-grained); DiffServ reserves for classes of flows; best effort makes no reservation. The Internet runs best effort: reservation needs an entity that owns the network, grants resources and enforces limits, and the Internet has no single owner.

Method.

1. Follow the layers bottom up: name resolution (DNS), transport connection (TCP), security (TLS), then the application request (HTTP).
2. For each step give protocol, transport and port, and what it produces for the next step.
3. For course questions, quote the slide's wording (conditions, features).

Formulas. No formula. Ports: DNS UDP/53 (hta3 p. 17); HTTP 80, HTTPS 443 (hta6 p. 12). Handshake order: TLS handshake after the TCP handshake (hta6 p. 14). QUIC: session and TLS handshake combined, 3-way not 6-way (hta6 p. 15). IntServ, DiffServ, best effort (hta1p p. 16).

Worked solution. DNS gives the IPv4 address needed to open TCP; the TCP 3-way handshake opens the connection to port 443; TLS checks the certificate and agrees the key; HTTP GET then travels encrypted and the 200 OK response returns the page. Certificate: issued to the site we are connected to, signed by a trusted CA (hta6 p. 13).

Check. Each step needs the result of the previous one (address, connection, key), so the order cannot change.

Marking and traps. (a) 2, 0.5 per exchange in the right order. (b) 1. (c) 1. (d) 0.5 for the three definitions, 0.5 for the Internet and the reason. **Traps.** Putting TLS before TCP; calling QUIC a kernel transport protocol (it is an application layer protocol over UDP); swapping IntServ (per flow) and DiffServ (per class).

1. Session 1: Protocol stack and statistical multiplexing (5 points) inspired by: hta1p p. 3 to 6 and 20; hta5 p. 7

A browser sends an HTTP request; it goes down through TCP, IPv4 and Ethernet.

(a) Name the 7 OSI layers and the 4 TCP/IP layers and match them. Give the PDU name at the transport, network, datalink and physical layers. (1.5 pts)

Final answer

OSI Application, Presentation, Session → TCP/IP Application; Transport → Transport; Network → Network; Datalink, Physical → Link. PDUs: Segment (transport), Packet (network), Frame (datalink), Bit (physical); Data at the application layer.

(b) In which order do the headers appear in the transmitted frame? What is encapsulation, and why is the UDP checksum, which covers fields of the IP header, called a layering violation? (1 pt)

Final answer

Ethernet header, IP header, TCP header, HTTP data (Ethernet may also add a footer). Encapsulation: each layer adds its own header (possibly a footer) and only touches its own header. The UDP checksum includes part of the IP header (addresses, length, protocol): one layer depends on another layer's header, which is a layering violation.

(c) Statistical multiplexing: $N = 400$ subscribers share a multiplexer; each is active with probability $p = 2\%$ at a given time, independently. What is the average number of active subscribers? Which distribution does the number X of active subscribers follow, and which approximation does the course use? With Chebyshev's inequality for $k = 3$ (take $\sigma = \sqrt{\lambda}$ for the Poisson distribution), which capacity L (an integer number of links) guarantees a blocking probability $\Pr(X > L)$ of at most 1/9? The exact Poisson computation gives $\Pr(X > L) \approx 0.16\%$ for that L: what do you conclude about Chebyshev? (2.5 pts)

Final answer

Average $Np = 8$. X is binomial ($N = 400$, $p = 0.02$); with large N and small p the course uses the Poisson distribution with $\lambda = Np = 8$. $\sigma = \sqrt{8} \approx 2.83$; $L = 8 + 3 \times 2.83 = 16.49$, so $L = \mathbf{17}$ links. Then $\Pr(X > 17) \le \Pr(|X - 8| \ge 3\sigma) \le 1/9 \approx 11\%$. The exact value, about 0.16%, is far smaller: Chebyshev does not depend on the distribution, so it overestimates and over-dimensions the multiplexer.

Method.

1. Stack questions: list the OSI layers top down, map them to the 4 TCP/IP layers, name the PDU of each layer.
2. Encapsulation: going down, each layer adds its header in front, so the lowest layer's header comes first on the wire.
3. Statistical multiplexing: mean Np; binomial distribution, approximated by Poisson with $\lambda = Np$; Chebyshev gives $L = \mu + k\sigma$ with probability bound $1/k^2$.

Formulas.

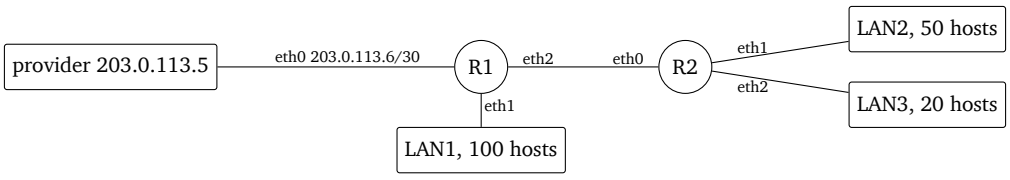
- OSI layers and PDUs (hta1p p. 5); TCP/IP layers (p. 6); encapsulation (p. 4).
- Binomial: $\mu = Np$; Poisson: $\lambda = Np$ for large N and small p; blocking $\Pr(X > L) = 1 - \mathrm{CDF}(X, L)$ (hta1p p. 20).
- Chebyshev: $\Pr(|X - \mu| \ge k\sigma) \le 1/k^2$ (hta1p p. 20). The slide writes $\sigma = \lambda$ for Poisson; the standard deviation is $\sigma = \sqrt{\lambda}$ [not in 2026 slides].

Worked solution. (c) $\mu = 400 \times 0.02 = 8$; $\lambda = 8$; $\sigma = \sqrt{8} \approx 2.83$; $3\sigma \approx 8.49$; $L = 8 + 8.49 = 16.49$, rounded up to 17 links (a number of links is an integer and must not be below the bound). $\Pr(X > 17) \le \Pr(X \ge 8 + 8.49) \le \Pr(|X - 8| \ge 3\sigma) \le 1/3^2 = 1/9 \approx 11\%$. The exact Poisson tail $1 - \sum_{x=0}^{17} e^{-8} 8^x / x!$ is about 0.16%.

Check. $1/9 \approx 0.111 \gg 0.0016$: the bound holds and is far from tight, as the slide says ("overestimates").

Marking and traps. (a) 1.5: layers 1, PDUs 0.5. (b) 1: order 0.5, layering violation 0.5. (c) 2.5: mean and distributions 1, $L = 17$ 1, conclusion 0.5. **Traps.** Dimensioning for 400 links; the slide writes $\sigma = \lambda$ (that is the variance), which would give $L = 8 + 24 = 32$: here the statement fixes $\sigma = \sqrt{\lambda}$; rounding 16.49 down to 16.

2. Session 2: Interface addresses and routing tables (5 points) inspired by: 2018 midterm, Q7; 2019 practice set, ex. 1



The company owns the block 192.168.40.0/24 for LAN1, LAN2, LAN3 and the R1-R2 link.

(a) Split the block from its start, largest need first: network/prefix of LAN1, LAN2, LAN3 and of the link. (1.5 pts)

Final answer

LAN1 100 hosts: /25 (126); LAN2 50: /26 (62); LAN3 20: /27 (30); link: /30. LAN1 **192.168.40.0/25**, LAN2 **192.168.40.128/26**, LAN3 **192.168.40.192/27**, link **192.168.40.224/30**.

(b) Give an address to every interface of R1 and R2 on these networks, and the default gateway of a PC on LAN3. (1 pt)

Final answer

One possible choice: R1 eth1 192.168.40.1/25, R1 eth2 192.168.40.225/30; R2 eth0 192.168.40.226/30, R2 eth1 192.168.40.129/26, R2 eth2 192.168.40.193/27. Default gateway of a LAN3 PC: 192.168.40.193, the interface of R2 on its LAN.

(c) Write the routing tables of R1 and R2 (destination/prefix, next hop or interface), including the default route. (1.5 pts)

Final answer

R1		R2	
192.168.40.0/25	eth1 (direct)	192.168.40.128/26	eth1 (direct)
192.168.40.224/30	eth2 (direct)	192.168.40.192/27	eth2 (direct)
203.0.113.4/30	eth0 (direct)	192.168.40.224/30	eth0 (direct)
192.168.40.128/26	via 192.168.40.226	0.0.0.0/0	via 192.168.40.225
192.168.40.192/27	via 192.168.40.226		
0.0.0.0/0	via 203.0.113.5		

R2's default route covers LAN1 and the Internet; an extra line 192.168.40.0/25 via 192.168.40.225 is also correct.

(d) Can R1 replace its routes to LAN2 and LAN3 with the single line 192.168.40.128/25? Follow a packet destined to 192.168.40.240. Why must R1 do NAT on eth0? (1 pt)

Final answer

It would not be an exact merge: /26 and /27 differ in size. 192.168.40.128/25 covers 128 to 255: LAN2, LAN3, the link (the directly connected /30 is more specific and wins) but also 228 to 255, which are unused. A packet to 192.168.40.240 would go from R1 to R2; R2 only has its default route and sends it back to R1: a loop until TTL = 0. Better keep the two lines. NAT: 192.168.0.0/16 is a private range, not routable on the Internet; R1 replaces the source address with its public address 203.0.113.6.

Method.
1. Size each network: smallest h with $2^h - 2 \geq$ hosts, prefix $32 - h$; a point-to-point link needs /30.
2. Allocate largest first from the start of the block.
3. Give each router interface a free host address of the network on that link; a PC's gateway is the router interface on its LAN.
4. Routing table: directly connected networks on their interface; remote networks via the neighbour's address on the shared link; a default route towards the exit.

Formulas.
- Hosts of a $/n$: $2^{32-n} - 2$ (hta2p p. 7).
- Nodes with the same prefix are directly reachable, others through the configured gateway (hta2 p. 6); route syntax "destination via next hop" (hta3p p. 14).
- Private ranges are not routable on the Internet, hence NAT (hta2p p. 8; hta5 p. 18).

Worked solution. 100: $2^6 - 2 = 62 < 100$, $2^7 - 2 = 126$: /25. 50: $2^6 - 2 = 62$: /26. 20: $2^5 - 2 = 30$: /27. Link: /30. LAN1 0 to 127 (128 addresses), LAN2 128 to 191 (64), LAN3 192 to 223 (32), link 224 to 227 (4); hosts of the link: .225 and .226. (d) 192.168.40.128/25 covers 128 to 255. R1 sends .240 to R2 (via .226); R2 has no specific line for .240, its default route returns it to R1 (.225): loop, TTL decreases by 1 at each hop until 0.

Check. $128 + 64 + 32 + 4 = 228 \leq 256$; addresses 228 to 255 (28) stay free, exactly the ones that would loop with the /25.

Marking and traps. (a) 1.5. (b) 1. (c) 1.5: remote networks via the neighbour's address on the shared link, default route. (d) 0.5 loop, 0.5 NAT. **Traps.** Using as next hop an R2 address that is not on the shared link (192.168.40.129); forgetting the return route on R2; giving LAN2 a /26 that overlaps LAN1.

3. Session 3: DHCP, DNS and ICMP (5 points)

inspired by: no past paper; hta3 p. 4 to 17; hta3p p. 18 to 20

A laptop with no network configuration joins a local network; the user then runs `traceroute www.example.org`.

(a) Describe the DHCP exchange: the four messages in order, which ones are broadcast and why, the transport and the ports, three pieces of information supplied besides the address. What happens if the lease is not renewed? (2 pts)

Final answer

DORA. Discover (client; link layer and IP broadcast: it has no address and does not know the server); Offer (a server offers an address; several servers may answer); Request (broadcast: it notifies all DHCP servers of the chosen offer); Acknowledgement (confirmation). UDP, server port 67, client port 68. TLV options: netmask, default gateway, DNS server (also time server, hostname). The lease is soft state: if it is not renewed it expires; using an expired address is a problem if the server assigns it to another host.

(b) How does the laptop obtain the IPv4 address of www.example.org: which server does it ask and how does it know that server, which record does it request, where can the answer come from? (1.5 pts)

Final answer

It asks its local DNS resolver, whose address it received from DHCP. It requests the A record (AAAA for IPv6), over UDP port 53. The request is forwarded along the tree, first upward then downward, until a resolver that is authoritative for the name or that has a cached response.

(c) Explain how traceroute discovers the routers on the path: IP header field used, ICMP messages involved. (1.5 pts)

Final answer

It sends probes with TTL = 1, then 2, 3… Each router decrements the TTL; the one where it reaches 0 drops the packet and sends back ICMP Time Exceeded (type 11), revealing its address. Probes are UDP to a random port by default, ICMP Echo Request with `-I`, TCP SYN with `-T` (tools slides). An Echo Request that reaches the destination gets an Echo Reply (type 0).

Method.
1. DHCP: write the four messages in order, who sends each, broadcast or not, and why.
2. DNS: who is asked (local resolver), how it is known (DHCP), which record, how the answer is found (tree, authoritative or cached).
3. Traceroute: increasing TTL, Time Exceeded from each router.

Formulas. No formula. DHCP over UDP, server port 67, client port 68 (hta3 p. 8); DORA (p. 10); records A, AAAA, MX (p. 14); lookup (p. 15); ICMP Time Exceeded type 11, Echo 8 and 0 (p. 4 and 6); traceroute probes (hta3p p. 20).

Worked solution. Discover (client to everybody), Offer (server to client), Request (client to everybody, so the other servers drop their offers), Acknowledgement (server). DNS: A record of www.example.org over UDP 53 to the resolver given by DHCP. Traceroute: TTL 1 expires at the first router, which returns Time Exceeded; TTL 2 at the second, and so on until the destination answers.

Check. Exactly 2 of the 4 DHCP messages are broadcast by the client; each TTL value reveals one more router.

Marking and traps. (a) 2: order of messages 1, broadcast and ports 0.5, options and lease 0.5. (b) 1.5. (c) 1.5: TTL 0.75, Time Exceeded 0.75. **Traps.** Thinking Request is unicast; mixing up the local resolver and the authoritative server; saying that the routers answer the probes with Echo Reply.

4. Session 4: One Bellman-Ford iteration (distance vector) (5 points)

inspired by: 2017 topic list, no. 15; hta4p p. 3 and 4

Vector from	A	B	C	D	E	F
A (cost 2)	0	4	6	5	8	10
B (cost 7)	4	0	2	1	3	9
C (cost 3)	6	2	0	7	2	4

Router X has three neighbours, A, B and C, with the link costs shown. It has just received their distance vectors (left, destinations other than X).

(a) Compute X's new table from these vectors only: for each destination A to F, distance and next hop, with the formula used. (2.5 pts)

Final answer

$d_X(Y) = \min_{v \in \{A,B,C\}} \big(c(X,v) + d_v(Y)\big)$, with $d_v(v) = 0$.

Dest.	via A ($2 + d_A$)	via B ($7 + d_B$)	via C ($3 + d_C$)	Distance	Next hop
A	2	11	9	2	A
B	6	7	5	5	C
C	8	9	3	3	C
D	7	8	10	7	A
E	10	10	5	5	C
F	12	16	7	7	C

B is reached via C (5), not over the direct link (7).

(b) Which vector does X advertise next? With split horizon, what does X's advertisement to C contain? (1 pt)

Final answer

X advertises (A 2, B 5, C 3, D 7, E 5, F 7), without next hops. Split horizon: X does not advertise to C the routes whose next hop is C (B, C, E, F); to C only A 2 and D 7.

(c) Compare distance vector and link state (information sent, recipients, algorithm, convergence), with one protocol of each family. Which protocol is used between autonomous systems, and of which type is it? (1.5 pts)

Final answer

DV: tell my neighbours who I know (destination, distance), to neighbours only, distributed Bellman-Ford, slow convergence and count to infinity; RIP, IGRP. LS: tell everybody who my neighbours are (LSA), to all known nodes, Dijkstra on the whole topology, faster reconfiguration after a link failure; OSPF, IS-IS. Between ASes: BGP, path vector (whole paths are exchanged), policy routing.

Method.

1. For each destination y, compute $c(X,v) + D_v(y)$ for every neighbour v.
2. Keep the minimum; the neighbour giving it is the next hop.
3. Advertise (destination, distance) without the next hop; with split horizon, leave out the routes whose next hop is the receiving neighbour.

Formulas.

- DV rule: a neighbour reaching y in $D_v(y)$ gives $c(X,v) + D_v(y)$ through it, choose the best (hta4 p. 7; hta4p p. 4).
- $D_X(y) = \min_v\{c(X,v) + D_v(y)\}$ [written form of the Bellman-Ford relaxation of hta4p p. 3].
- Split horizon (hta4p p. 6).

Worked solution. A: $2+0=2$, $7+4=11$, $3+6=9$: 2 via A. B: $2+4=6$, $7+0=7$, $3+2=5$: 5 via C. C: $2+6=8$, $7+2=9$, $3+0=3$: 3 via C. D: $2+5=7$, $7+1=8$, $3+7=10$: 7 via A. E: $2+8=10$, $7+3=10$, $3+2=5$: 5 via C. F: $2+10=12$, $7+9=16$, $3+4=7$: 7 via C.

Check. No destination has a tie for the minimum; the next hops via C are exactly the four routes split horizon removes towards C.

Marking and traps. (a) 2.5: formula 0.5, table 2. (b) 1. (c) 1.5. **Traps.** Keeping the direct link to B by reflex; adding the link cost twice; putting the next hop in the advertisement.

5. Session 5: TCP connection, fast retransmit and window size (5 points)

inspired by: 2020 exams; hta5 p. 9, 10; hta5p p. 13, 16

(a) A client opens a TCP connection with initial sequence number $S_c = 7300$; the server picks $S_s = 1200$. Give the flags, Sequence and Acknowledgement numbers of the three handshake segments. Which attack targets the server while it waits for the last segment, and which defense does the course give? (1.5 pts)

Final answer

SYN (seq 7300); SYN,ACK (seq 1200, ack 7301); ACK (seq 7301, ack 1201). SYN flood: lots of SYN segments with no intention to connect exhaust the connection table, where the server remembers the sequence numbers it has to verify. Defense: SYN cookies; the server does not remember its chosen sequence number but generates it with a cryptographic function and checks the client's ACK with the same function.

(b) As on the slides, numbers count segments. The sender sends segments 40 to 45; segment 42 is lost; the receiver acknowledges every segment it receives. List the ACKs. When does Reno retransmit, which ACK follows the arrival of the retransmitted 42, and what does Reno do to cwnd? (1.5 pts)

Final answer

40: ACK 41; 41: ACK 42; 43: ACK 42 (1st duplicate); 44: ACK 42 (2nd); 45: ACK 42 (3rd duplicate): fast retransmit of 42. When 42 arrives: **ACK 46**, cumulative, since 43 to 45 were already received. Fast recovery: cwnd/2 and stay in congestion avoidance, no slow start.

(c) A 200 Mbit/s link has a 60 ms RTT. Which window, in octets, is needed to use it fully? Which maximum throughput does the Window Size field alone allow (16 bits, counted in octets)? Which minimum window scale shift is needed? (2 pts)

Final answer

Window $\geq$ rate $\times$ RTT $= 200 \cdot 10^6 \times 0.060 = 12 \cdot 10^6$ bits = **1.5 MB** (1,500,000 octets). Window Size alone: 65,535 octets per RTT, so $65{,}535 \times 8/0.060 \approx$ **8.74 Mbit/s**, 4.4% of the link. Window scale: $65{,}535 \times 2^4 = 1{,}048{,}560$ is not enough, $65{,}535 \times 2^5 = 2{,}097{,}120 \geq 1{,}500{,}000$: shift **5** (maximum 14).

Method.

1. Handshake: SYN carries Sc; SYN,ACK carries Ss and acknowledges Sc+1; ACK carries Sc+1 and acknowledges Ss+1.
2. ACKs: the receiver always acknowledges the next segment it is waiting for; a gap repeats the same ACK; the third duplicate triggers the retransmis-

sion.
3. Window: needed window = rate × RTT in octets; compare with the 16-bit maximum; find the smallest shift s with $65{,}535 \times 2^s$ large enough.

Formulas.

- SYN (Sc); SYN,ACK (Ss, Sc+1); ACK (Sc+1, Ss+1) (hta5 p. 9).
- Acknowledgement = next segment to be received (hta5 p. 11); 3 duplicate ACKs: fast retransmit (hta5p p. 13), cwnd/2 and stay in congestion avoidance (p. 14).
- 16-bit window, shift up to 14 bits (hta5 p. 16; hta5p p. 16).
- Window = rate × RTT; throughput ≤ window / RTT [not in 2026 slides].

Worked solution. (b) 40 arrives: next expected 41. 41: 42. 43: still 42 (1st duplicate). 44: 42 (2nd). 45: 42 (3rd): retransmit 42. 42 arrives, 43 to 45 are buffered: next expected 46. (c) $200 \times 10^6 \times 0.060 = 12 \times 10^6$ bits; /8 = 1,500,000 octets. $65{,}535 \times 8 = 524{,}280$ bits per RTT; /0.060 = 8,738,000 bit/s ≈ 8.74 Mbit/s; 8.74/200 ≈ 4.4%. $2^4 = 16$: $65{,}535 \times 16 = 1{,}048{,}560 < 1{,}500{,}000$; $2^5 = 32$: 2,097,120 ≥ 1,500,000: $s = 5$.

Check. 1,500,000/65,535 ≈ 22.9, between 2^4 and 2^5.

Marking and traps. (a) 1.5: segments 1, attack and defense 0.5. (b) 1.5: ACK list 0.5, moment of retransmission 0.5, ACK 46 and cwnd 0.5. (c) 2: bandwidth-delay product 1, throughput without the option 0.5, shift 0.5. **Traps.** ACK S_c instead of $S_c + 1$; retransmitting at the first duplicate ACK (that is New Reno); answering ACK 43 after the retransmission; mixing up bits and octets; using the one-way delay instead of the RTT.

6. Session 6: E-mail (5 points)

inspired by: no past paper; hta6p p. 4 to 18

alice@example.org sends, from her mail client, a message with a PDF attachment to bob@example.net, who reads his mail on his phone and on his computer.

(a) Describe the path of the message: agents crossed (MUA, MTA, MDA), protocol and port at each step, and how the MTA of example.org finds the one of example.net. (2 pts)

Final answer

Alice's MUA submits the message with SMTP on the submission port 465 (formerly 587), with TLS and authentication, to the MTA of example.org. That MTA looks up the MX record of example.net in DNS and relays with SMTP, port 25, to the MTA of example.net. The MDA of example.net (a relay that can store messages) puts it in Bob's mailbox. Bob's MUA retrieves it with IMAP (143, 993 with TLS) or POP3 (110, 995).

(b) Why should Bob use IMAP rather than POP3? (1 pt)

Final answer

IMAP queries the message database of the server: multiple mailboxes in directories, headers and attachments downloaded separately, browsing without retrieving; both devices see the same state. POP3 is simple retrieval of the inbox only, optionally deleting from the server, no subfolders.

(c) The message format only allows 7-bit ASCII: how is the attachment carried? (1 pt)

Final answer

MIME converts it to 7-bit ASCII (base64 or quoted-printable encoding), labels it with `Content-Type: application/pdf`, and puts text and attachment in a multipart body.

(d) How can the MTA of example.net check that the message comes from a server authorized for example.org? Name two mechanisms and their principle. (1 pt)

Final answer

SPF: the MTA proves that it is authorized to send mail for the domain in the From header. DKIM: the sender MTA signs the mail, with its public keys in DNS. (DMARC publishes in DNS how the domain's MTA is set up for DKIM and SPF.)

Method.

1. Draw the chain MUA → MTA → MTA → MDA → MUA and label each arrow with protocol and port.
2. Name how the next MTA is found (MX record).
3. For POP3 vs IMAP, MIME, SPF and DKIM, give the slide's one-line principle.

Formulas. No formula. SMTP 25, submission 465 (formerly 587) (hta6p p. 7); POP3 110, 995; IMAP 143, 993 (p. 8, 9); MIME (p. 5); DKIM, SPF, DMARC (p. 18).

Worked solution. Submission with SMTP and authentication; relay with SMTP between MTAs after the MX lookup; storage by the MDA; retrieval by Bob's MUA with IMAP or POP3. Attachment: base64 (or quoted-printable) in a multipart body with its Content-Type.

Check. SMTP only pushes mail towards the mailbox; reading always uses POP3 or IMAP.

Marking and traps. (a) 2: chain of agents 1, ports 0.5, MX 0.5. (b) 1. (c) 1. (d) 0.5 per mechanism. **Traps.** Retrieving mail with SMTP; saying that Alice's MUA contacts Bob's server directly; mixing up DKIM (signature) and SPF (authorization of the sending MTA).